

Kingdom of Saudi Arabia
Ministry of Education
Shaqra University
College of Computing and Information Technology
Computer Science Department



Electro Riyal

Blockchain Network Simulation System

Cryptography Project

2025-2026

Students:

Aljawharah Alsaqan
Metebah Al-dawsri
Raghad Fayez
Hanan Abdallah

ID:

443500088
443500038
444500629
443520329

Supervisor:

Dr.Samia Dardory

Abstract

This project presents the development of Electro Riyal, a secure blockchain-based transaction system that utilizes cryptographic principles to ensure data integrity, immutability, and decentralized consensus. The backend is implemented using Flask, while the frontend is built using React and Material-UI. This document explains the cryptographic foundations behind the system, including hashing, Proof of Work, and chain validation. The design, architecture, and implementation details are provided along with an analysis of how cryptography ensures security within the blockchain.

Table of Contents

Topic	page
1. Introduction	4
2. Problem Statement	4
3. Objective	4
4. Motivation	4
5. Background	5
6. Methodology	6
7. System Design	7
8. Implementation Details	7
9. Results and Discussion	9
10. Conclusions and Future Work	11
References	12

Table of Figures

Figure	Title	page
Figure 1	The main interface of the Electro Riyal system the device	9
Figure 2	Adding a new transaction by entering sender, receiver, and amount.	10
Figure 3	Viewing the updated blockchain after adding and mining the new block.	10

1. Introduction

Cryptography plays a central role in securing modern distributed systems. Blockchain technology, in particular, relies heavily on hashing algorithms, digital signatures, and decentralized consensus models to create tamper-proof records of transactions. This project implements a simplified blockchain system that demonstrates the core principles of cryptography in action.

The goal of *Electro Riyal* is to simulate a secure digital currency network where data cannot be modified, blocks must be mined using computational work, and nodes maintain a consistent ledger.

2. Problem Statement

Traditional centralized databases are vulnerable to manipulation, single points of failure, and unauthorized access. There is a growing need for systems where:

- Data cannot be altered retroactively
- Every change is verifiable
- No single entity controls the records
- Security is ensured using cryptographic methods

Blockchain solves this problem by using **cryptographic hashing** and **distributed consensus** to maintain a secure ledger.

3. Objectives

The main objective of this project is to combine technology and creativity to provide a joyful and engaging experience that allows users to transform real photos into SpongeBob-style cartoon images.

The specific objectives are:

1. To design a simple web application that allows photo upload and direct camera capture.
2. To apply classical image processing techniques such as bilateral filtering and edge detection.
3. To make the process fun, fast, and accessible for all users.
4. To encourage creativity and self-expression through interactive technology.

4. Motivation

This project aims to serve as an educational model that demonstrates how cryptographic concepts—such as hashing, Proof of Work, and decentralization—are used to build a secure and tamper-resistant blockchain system. It helps students understand the practical application of cryptography and how it protects data in modern digital systems.

5. Background

5.1 Hashing

Hashing is a one-way cryptographic process that converts input data into a fixed-length output known as a hash value. The *Electro Riyal* system uses **SHA-256**, a widely adopted hashing algorithm used in blockchain systems such as Bitcoin[1], [2].

Cryptographic hash functions provide several essential properties:

- **Deterministic:** The same input always produces the same hash.
- **Collision-resistant:** It is extremely difficult to find two different inputs that result in the same hash value.
- **Non-reversible:** It is computationally infeasible to reconstruct the original input from the hash.
- **Fast to compute:** Hashes can be generated quickly even for large inputs.

In this project, SHA-256 is used to compute the hash of each block, ensuring **data integrity and immutability**.

```
python

hashlib.sha256(data.encode()).hexdigest()
```

5.2 Proof of Work (PoW)

Proof of Work (PoW) is a consensus mechanism that makes block creation intentionally computationally expensive to prevent malicious manipulation [2], [3].

To create a new block, the miner must find a **proof value** such that the resulting block hash satisfies a predefined pattern (such as starting with a specific number of zeros). This computational difficulty ensures that if an attacker attempts to modify a block, they must redo the PoW for that block and all blocks that follow—making tampering practically impossible.

```
csharp

sha256(new_proof2 - previous_proof2) starts with '00000'
```

5.3 Block Structure

Each block in the blockchain contains the following fields:

- **index**
- **timestamp**
- **proof**
- **previous_hash**
- **transactions**
- **hash**

The block's hash is computed from all of these components. Therefore, any modification to the block's contents—even a minor one—produces a completely different hash, ensuring **immutability and secure data linking** [1].

5.4 Chain Validation

To maintain integrity across the blockchain, each node performs **chain validation**, which includes:

- Verifying that each block's **previous_hash** field matches the hash of the preceding block.
- Confirming that the **Proof of Work** is correct and meets the required difficulty level.
- Ensuring that no tampering or unauthorized modifications have occurred.

These validation steps guarantee that the blockchain remains **consistent, tamper-resistant, and secure** [2], [3].

6. Methodology

The project follows a **structured development approach**:

1. **Requirement Analysis:** Define secure transaction handling, tamper-proof storage, and distributed consensus.
2. **Cryptography Integration:**
 - SHA-256 hashing for block integrity.
 - Proof of Work to prevent tampering.
 - Chain validation to ensure consistency.
3. **Backend Development:** Implement blockchain logic, REST APIs, and node synchronization.
4. **Frontend Development:** Create a responsive UI for blockchain interaction.
5. **Testing & Validation:** Unit and integration testing to verify correctness, immutability, and PoW compliance.
6. **Documentation & Analysis:** Record design decisions and evaluate cryptographic security.

7. System Design

The Electro Riyal system uses a **two-tier architecture**:

Backend (Flask API):

- Manages blockchain operations: block creation, Proof of Work, hashing, chain validation.
- Handles transactions and node communication.
- Provides RESTful APIs for mining (/mine_block), adding transactions (/add_transaction), chain retrieval (/get_chain), validation (/is_valid), and node management (/connect_node, /replace_chain).

Frontend (React + Material-UI):

- Offers a clean, interactive interface for mining, adding transactions, and viewing the blockchain.
- Uses React Router for navigation and Material-UI for theming.

Blockchain Structure:

Each block includes index, timestamp, proof, previous_hash, transactions, and hash. Blocks are linked through SHA-256 hashes to ensure immutability.

8. Implementation Details

8.1 Core Blockchain Class

The Blockchain class handles:

- chain initialization
- block creation
- hashing
- Proof of Work
- validating chain consistency
- managing nodes
- replacing the chain

Example: Block creation

```
python

def create_block(self, proof, previous_hash):
    block = {
        "index": len(self.chain) + 1,
        "timestamp": str(datetime.datetime.now()),
        "proof": proof,
        "previous_hash": previous_hash,
        "transactions": self.transactions,
    }
```

8.2 Proof of Work Implementation

The system continuously tests proofs until a hash starting with '00000' is found:

```
while not valid:
    hash_operation = sha256(str(new_proof**2 - previous_proof**2)).hexdigest()
    if hash_operation[:5] == '00000':
        valid = True
```

This creates a computational challenge.

8.3 Flask Endpoints

- **Mine Block:**
Creates a block after solving PoW
- **Validate Chain:**
Ensures no tampering
- **Add Transaction:**
Stores transactions until next mining
- **Connect Node:**
Registers new blockchain nodes
- **Replace Chain:**
Prevents forks by adopting the longest valid chain

8.4 React User Interface

Key features of the UI:

- Modern interface using Material-UI Theme
- Custom color palette
- Clean navigation using React Router
- Landing screen with branding
- Button-based blockchain interactions

9. Results and Discussion

The system successfully demonstrates:

- How hashing secures block data
- How Proof of Work prevents tampering
- How cryptography ensures immutability
- How nodes maintain distributed consensus
- How users can interact through a web interface

When testing the API, each mined block produces:

- a valid PoW
- a correct previous hash
- a unique SHA-256 hash

This confirms the correct implementation of basic blockchain cryptography.



Figure1: The main interface of the Electro Riyal system

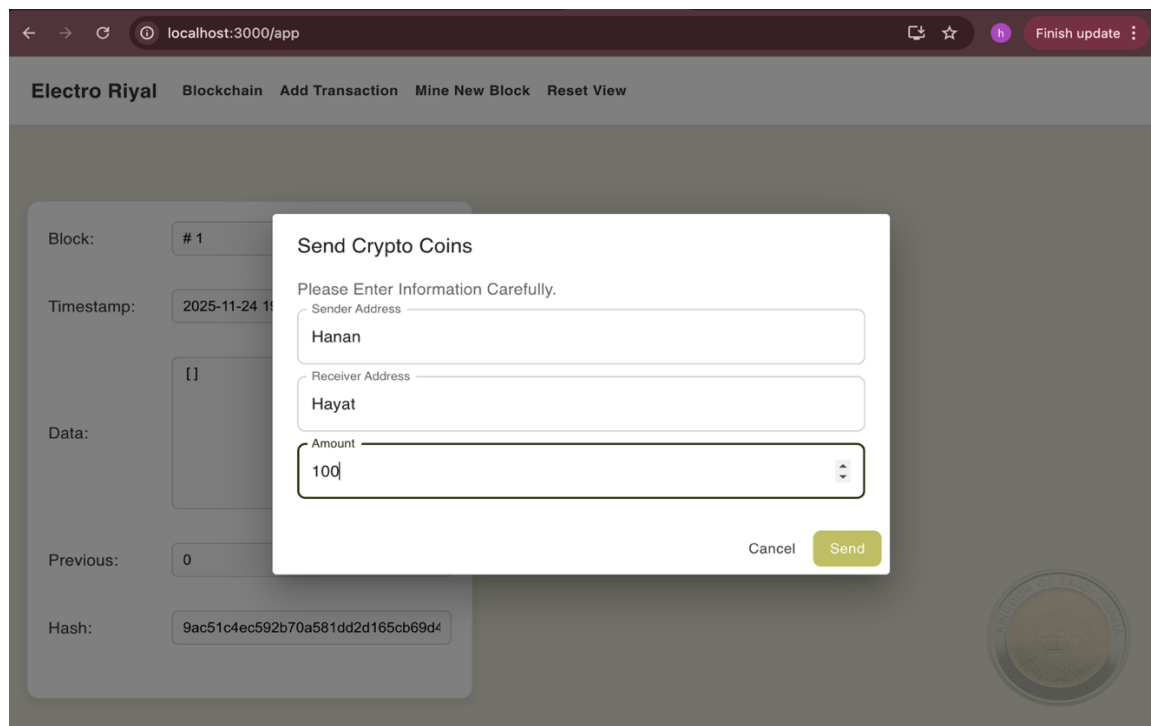


Figure2: Adding a new transaction by entering sender, receiver, and amount.

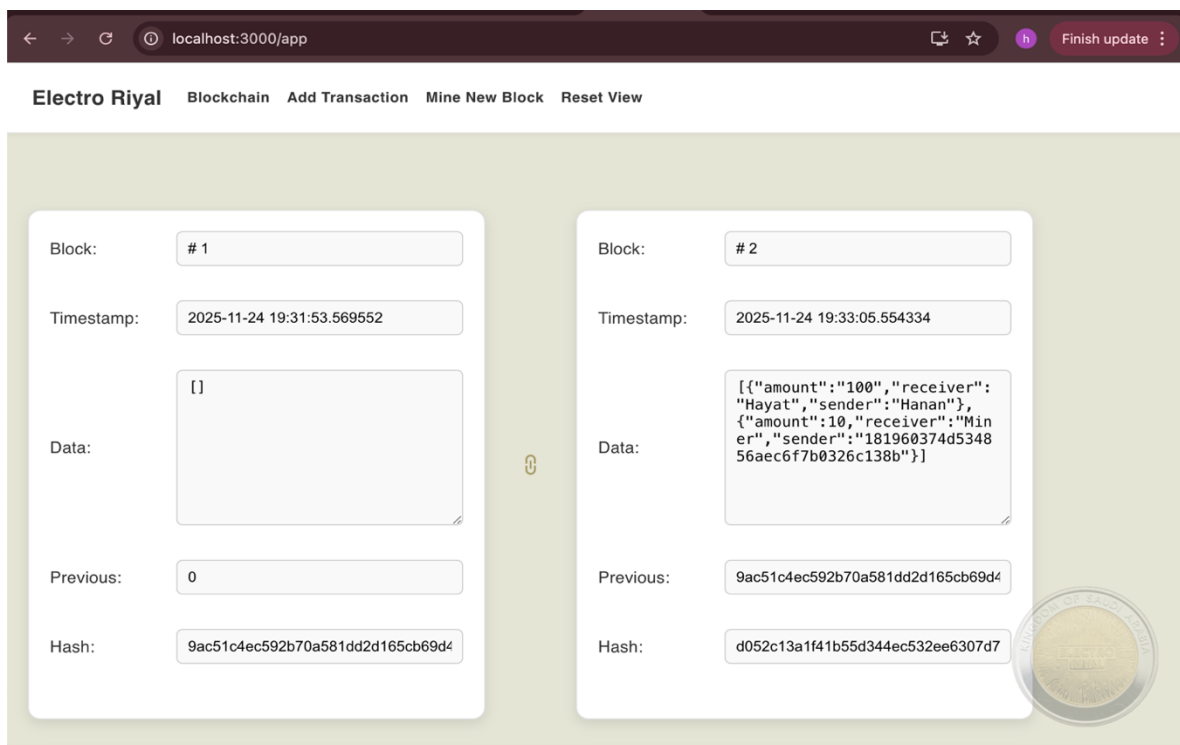


Figure 3: Viewing the updated blockchain after adding and mining the new block.

10. Conclusions and Future Work

10.1 Conclusion:

This project illustrates how blockchain technology uses cryptography to construct secure, transparent, and tamper-resistant systems. By implementing hashing, Proof of Work, and distributed validation, the system demonstrates the core mechanisms behind secure digital currencies.

The Electro Riyal blockchain is a simplified but fully functional model showcasing cryptographic security in practice.

10.2 Future Work:

Future improvements could include:

- Implement digital signatures (ECDSA)
- Add a wallet system
- Store chain data persistently
- Improve UI dashboards
- Add block encryption for enhanced privacy
- Implement more advanced consensus algorithms

References

- [1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>. [Accessed: Nov. 4, 2025].

- [2] W. Stallings, Cryptography and Network Security: Principles and Practice, 7th ed. Pearson, 2017. [Online]. Available: <https://doi.org/10.5555/3154482>. [Accessed: Nov. 4, 2025].

- [3] A. Narayanan, J. Bonneau, E. Felten, A. Miller, and S. Goldfeder, Bitcoin and Cryptocurrency Technologies. Princeton University Press, 2016. [Online]. Available: <https://doi.org/10.2307/j.ctv346nd>. [Accessed: Nov. 4, 2025].

- [4] Python Software Foundation, “hashlib — Secure hashes and message digests,” Python Documentation, 2025. [Online]. Available: <https://docs.python.org/3/library/hashlib.html>. [Accessed: Nov. 4, 2025].

- [5] M. Crosby, N. Pattanayak, S. Verma, and V. Kalyanaraman, “Blockchain technology: Beyond bitcoin,” Applied Innovation Review, vol. 2, pp. 6–19, Jun. 2016. [Online]. Available: <https://j2-capital.com/wp-content/uploads/2017/11/AIR-2016-Blockchain.pdf>. [Accessed: Nov. 4, 2025].